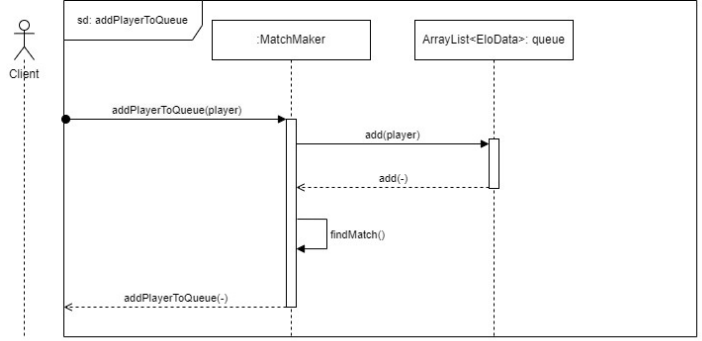
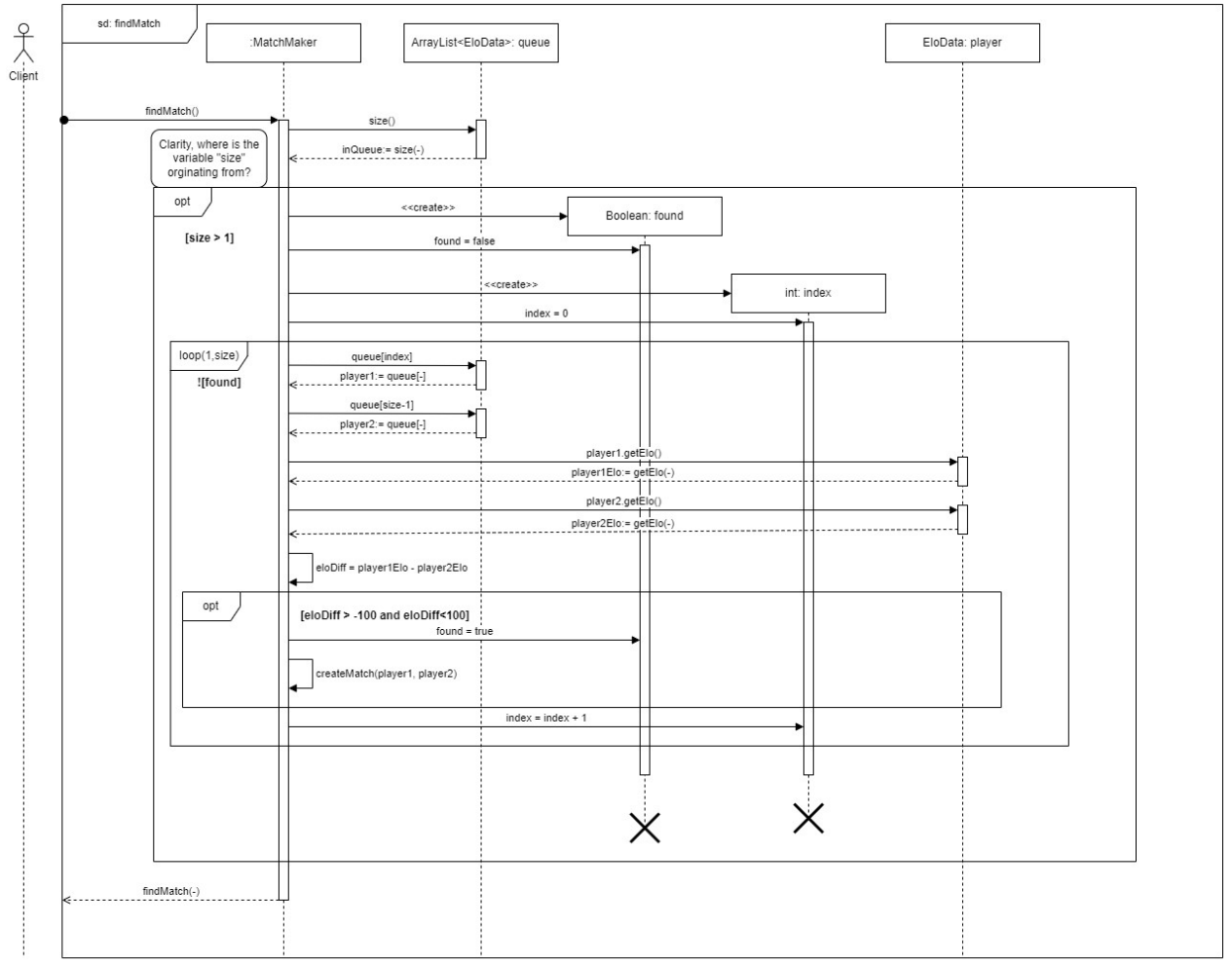
Overall: No diagram errors, just suggestions to possible enhance user experience.



Suggestion: Add a message indicating a player has been added to the queue, and a match is trying to be found? For example "You have been added to the Queue for [GAME]!"

Overall: No diagram errors, but there could be improvements in the code based on the diagram.



Optimization: player1 could be defined before entering the loop, rather than being re-defined for each iteration. This can drecrease resource consumption in the case of high player count in the queue.

Suggestion: Expand the boundary by a set amount if no player is within 100 elo. For example, after no elo of 100 is calculated, increase the boundary by 50 elo to find a player within 150 elo (keep growing until a player is found).

It would be inefficient if two players were only in the queue, but since their elo is not within 100 of each other, they are not paired, which creates player frustration and can decrease player count as players may be paired less frequently. However, there must be a mechanism that handles elo calculation due to widening elo differences, which suggests different skill levels between two players.

This can be implemented by having a set constant that increases the bound each time. To decrease runtime, have an array of all the calculated elo of the players already in the queue relative to the player.

Optimization: Instead of considering the negative and positive of the bound, use Math.abs(eloDiff). Do note you are importing an extra module, but using Math.abs(eloDiff) improves code readability.

Suggestion: If, for an unknown reason (such as game crashing, or not enough players), no match has been found, add a message that informs the player no match has been found. No message can easily confuse a game player waiting in queue.

Overall: No diagram errors.

